CineMatch

MovieLens 20M Hybrid Recommender System

PROJECT REPORT

Dataset	MovieLens 20M (20M ratings · 138K users · 27K movies)
Architecture	TF-IDF Content-Based + SVD Collaborative + Hybrid Blend
Key Innovation	Genome-tag enrichment + MMR Diversity Re-ranking
Evaluation	8 metrics: P@K · R@K · NDCG · MRR · Hit Rate · Diversity
UI	5-tab Streamlit app with real-time controls & Plotly charts
Date	2023

Table of Contents

1. **Executive Summary** 3
2. **Problem Statement & Motivation** 4
3. **Dataset — MovieLens 20M** 5
   - 3.1 Dataset Files & Schema 5
   - 3.2 Genome Tags — The Rich Content Signal 6
   - 3.3 Data Preprocessing Pipeline 7
   - 3.4 Bayesian Rating Smoothing 8
4. **Content-Based Filtering** 9
   - 4.1 Core Concept 9
   - 4.2 TF-IDF Vectorization — Deep Dive 9
   - 4.3 The Metadata Soup Strategy 12
   - 4.4 Cosine Similarity 13
   - 4.5 Sparse Nearest-Neighbour at Scale 14
   - 4.6 Profile-Based Recommendations 15
5. **Collaborative Filtering** 16
   - 5.1 Core Concept 16
   - 5.2 The User-Item Matrix 17
   - 5.3 Matrix Factorization via Truncated SVD 18
   - 5.4 User Mean-Centering 20
   - 5.5 Item-Item Latent Similarity 21
   - 5.6 Cold-Start Handling 22
6. **Hybrid Recommendation Engine** 23
   - 6.1 Why Neither Approach Alone Is Sufficient 23
   - 6.2 Score Normalization 24
   - 6.3 Weighted Hybrid Blend Formula 24
   - 6.4 Popularity Signal & Bayesian Boost 25
7. **MMR Diversity Re-Ranking** 26
   - 7.1 The Filter Bubble Problem 26
   - 7.2 Maximal Marginal Relevance — Theory 26
   - 7.3 Jaccard Genre Similarity 27
   - 7.4 MMR Algorithm Step-by-Step 28
8. **System Architecture & File Structure** 29
9. **Streamlit User Interface** 31
10. **Evaluation Framework** 32
    - 10.1 Offline Hold-Out Protocol 32
    - 10.2 All Eight Metrics Explained 33
    - 10.3 Expected Results & Benchmarks 36
11. **Performance, Memory & Scalability** 37
12. **Limitations & Future Work** 38
13. **Conclusion** 39

1. Executive Summary

CineMatch is a production-grade movie recommendation system trained on the MovieLens 20M dataset — one of the most widely used benchmarks in recommender systems research, containing 20 million explicit ratings given by 138,000 users to 27,000 films over several decades. The system combines three complementary machine learning paradigms into a single, unified pipeline: content-based filtering driven by genome-tag-enriched TF-IDF representations, collaborative filtering via Singular Value Decomposition (SVD) matrix factorisation, and a configurable hybrid blending engine with Maximal Marginal Relevance (MMR) diversity re-ranking.

Project at a Glance	
Dataset	: MovieLens 20M — 20,000,263 ratings · 138,493 users · 27,278 movies
Content Model	: TF-IDF (unigram + bigram, 20K vocab) on genome-tag enriched soup
Collab Model	: Truncated SVD with k=100 latent factors, user mean-centred ratings
Hybrid Formula	: score = α·content + (1−α)·CF + 0.1·Bayesian_popularity
Diversity	: MMR re-ranking with Jaccard genre dissimilarity (λ = 0.3)
Evaluation	: 8 metrics: Precision, Recall, F1, NDCG, MRR, Hit Rate, Diversity, Novelty
UI	: 5-tab Streamlit app — Similar · Watchlist · Trending · Analytics · Eval
Performance	: Full pipeline loads in ~15 min; Parquet cache cuts restarts to ~10 sec

The system is engineered to address the three canonical problems in recommendation: the cold-start problem (new users with no history), the filter bubble problem (over-specialised recommendations within a single genre), and the scalability problem (20M rows of ratings cannot be processed naïvely). Each engineering decision in this codebase — from chunked CSV loading to sparse dot-product similarity to Bayesian smoothing — directly addresses one of these challenges.

This report provides a complete technical explanation of every concept, formula, design decision, and trade-off in the system, written to be comprehensible to anyone with a working knowledge of Python and basic linear algebra.

2. Problem Statement & Motivation

Modern streaming platforms catalogue tens of thousands of titles. Without personalisation, discovering content is either random (browse) or driven by marketing (promoted). Neither serves users well. A recommendation system solves this by predicting which films a specific user is most likely to enjoy — transforming an overwhelming catalogue into a curated personal list.

2.1 The Three Core Challenges

Every recommendation system must navigate three fundamental tensions:

Challenge	Description & Why It Is Hard
Information Overload	With 27,000 movies, a user would need over 4 years of continuous watching to see everything. The system must narrow this to a handful of highly relevant suggestions in milliseconds.
Cold-Start Problem	A brand-new user has no rating history. Collaborative filtering — which learns from past behaviour — cannot help them at all. Content-based filtering is the only option until enough ratings accumulate.
Filter Bubble / Diversity	A system that only recommends more of what the user already likes creates an echo chamber. Pure relevance maximisation produces lists where all 10 recommendations are from the same genre. Users report these as boring.
Sparsity	Even the most active MovieLens users have rated fewer than 1% of the catalogue. The user-item matrix is ~98.9% empty. Collaborative methods must generalise from a tiny fraction of observed data.
Popularity Bias	Systems naïvely trained on ratings will recommend blockbusters (which have millions of ratings) over equally good niche films (which have hundreds). This gives users nothing they couldn't find themselves.
Scalability	20 million rows of CSV cannot be loaded into a Pandas DataFrame naïvely without careful chunking, dtype optimisation, and caching. SVD on a 138K × 27K matrix requires sparse linear algebra to be tractable.

2.2 Design Principles

CineMatch is built around four core design principles that directly address the challenges above:

- Complementarity: Use content-based and collaborative methods together so each covers the other's blind spots.
- Configurability: The alpha parameter lets users tune the content-collaborative balance in real time, and the MMR lambda controls diversity vs. relevance.
- Graceful degradation: When collaborative signals are absent (cold start), the system falls back to content-based without breaking.
- Engineering pragmatism: Every algorithmic choice is constrained by what is feasible on a standard laptop — chunked loading, sparse matrices, Parquet caching, and on-demand similarity (not a pre-stored full matrix).

3. Dataset — MovieLens 20M

The MovieLens 20M dataset, published by the GroupLens research group at the University of Minnesota, is one of the most widely cited benchmarks in recommender systems research. It contains explicit ratings (half-star increments from 0.5 to 5.0) collected from real users of the MovieLens platform between January 1995 and March 2015.

3.1 Dataset Files & Schema

The dataset consists of six CSV files, each serving a distinct purpose in the pipeline:

File	Rows	Purpose & Key Columns
movie.csv	27,278	Movie metadata. Columns: movieId, title (includes year in parentheses), genres (pipe-separated list of genre tags). This is the primary content catalog.
rating.csv	20,000,263	User ratings. Columns: userId, movieId, rating (0.5–5.0), timestamp (Unix epoch). The central data source for collaborative filtering.
genome-scores.csv	11,709,768	Tag relevance scores. Columns: movieId, tagId, relevance (continuous 0–1). For each of ~10,381 movies, a relevance score for each of 1,128 tags. This is the richest content signal.
genome-tags.csv	1,128	Tag name lookup. Columns: tagId, tag (human-readable string like 'visually stunning', 'plot twist', 'based on a book').
tag.csv	465,564	User-generated tags. Columns: userId, movieId, tag (free-text string), timestamp. Crowd-sourced community tags applied by real users.
links.csv	27,278	Cross-reference IDs. Columns: movieId, imdbId, tmdbId. Links each MovieLens movie to its IMDB and TMDB entries for potential external enrichment.

3.2 Genome Tags — The Rich Content Signal

The genome tag system is the most distinctive and valuable feature of MovieLens 20M. It was created by crowdsourcing human judgements about which descriptive tags apply to which films, then training a model to produce continuous relevance scores for every (movie, tag) pair — including movies that no human explicitly tagged.

This produces a dense 10,381 × 1,128 matrix of continuous relevance scores where every cell is filled (unlike the sparse rating matrix). A relevance score of 0.95 for the tag 'visually stunning' applied to 'Mad Max: Fury Road' indicates that virtually all humans who engaged with both this film and this tag agreed it applies. A score of 0.02 for 'romantic' applied to the same film indicates near-unanimous disagreement.

Why Genome Tags Outperform Genre Tags for Content Similarity
Genre tags: 19 binary categories (Action, Drama, Comedy, ...). Extremely coarse.
Two films can share 'Drama' but be completely dissimilar in tone and theme.
Genome tags: 1,128 continuous-valued semantic descriptors per film.
'psychological', 'nonlinear timeline', 'unreliable narrator', 'bleak ending'
These capture nuance that genre alone cannot — 'dark comedy' vs 'romantic comedy'

	are both 'Comedy' by genre but are opposites by genome tag profile.
Result:	TF-IDF on genome tags finds films that feel similar, not just share a genre label.

In the pipeline, the top 1,000 genome tags are selected by average relevance across all movies (this removes tags that are universally near-zero). For each movie, the top 20 tags by relevance score are extracted and converted to text strings for TF-IDF processing.

3.3 Data Preprocessing Pipeline

The raw data cannot be used directly. The loader.py module applies a sequence of preprocessing steps before any model sees the data:

3.3.1 Chunked Loading of rating.csv

At 20 million rows, rating.csv is too large to load as a single Pandas read_csv() call on most machines. CineMatch reads it in chunks of 2,000,000 rows, concatenates them, and immediately frees each chunk from memory after concatenation. This keeps peak RAM usage bounded regardless of file size.

3.3.2 Memory-Efficient dtypes

Pandas defaults to int64 (8 bytes) for integer columns and float64 (8 bytes) for floats. CineMatch explicitly casts all IDs to int32 (4 bytes) and all ratings to float32 (4 bytes). For 20M ratings this alone saves approximately 240MB of RAM — the difference between fitting in 8GB and not.

3.3.3 Quality Filtering

Two filters are applied iteratively until convergence (typically 2–3 passes) to remove noise:

- Minimum ratings per user: Users who have rated fewer than 20 films are dropped. Their sparse histories provide insufficient signal for collaborative learning and may represent one-time visitors.
- Minimum ratings per movie: Films with fewer than 10 ratings are dropped. These are typically obscure titles for which collaborative signals cannot be computed reliably. Content-based recommendations can still find them.

The iteration is necessary because dropping users may cause some movies to fall below the movie threshold, and vice versa. After convergence, all remaining users and movies satisfy both constraints simultaneously.

3.3.4 Parquet Caching

After the first load, the filtered ratings DataFrame is saved to a Parquet file (binary columnar format). Parquet is approximately 5-10x faster to read than CSV and compresses the data to roughly 30% of the CSV file size. Subsequent application starts skip the 45-second CSV parse and load from Parquet in approximately 3 seconds.

3.4 Bayesian Rating Smoothing

The raw mean rating of a film is an unreliable quality signal when the number of ratings is small. A film rated 5.0 stars by 3 users appears to be better than The Shawshank Redemption in a naive ranking. Bayesian smoothing corrects this by shrinking each film's mean toward the global mean, weighted by how many ratings it has received:

```
bayesian_rating(m) = ( n(m) × mean(m) + C × global_mean ) / ( n(m) + C )
```

Where n(m) is the number of ratings for movie m, mean(m) is its raw mean rating, global_mean is the mean across all movies and all ratings, and C is a smoothing constant set to the 25th percentile of rating counts. When n(m) is very large, the Bayesian rating converges to the raw mean. When n(m) is small, it is pulled toward the global mean, penalising low-confidence estimates.

This Bayesian average is used for all popularity rankings, trending queries, and as the popularity component in the hybrid score. It is significantly more robust than raw means for a catalogue with highly variable numbers of ratings per film.

4. Content-Based Filtering

4.1 Core Concept

Content-based filtering recommends items based on the intrinsic properties of those items — their genres, themes, tone, director, cast, and any other descriptive attributes available. It does not require data from any other user. The fundamental assumption is: if a user liked Item A, they will probably like Item B if B is similar to A in its properties.

Content-based filtering is analogous to a librarian who says: 'You loved this thriller with an unreliable narrator? Let me find other thrillers that also feature unreliable narrators.' The librarian doesn't need to know what any other reader enjoyed — only the properties of the books themselves.

Strengths and Weaknesses of Content-Based Filtering
STRENGTHS:
• Works immediately for new users with no rating history (cold-start resistant)
• Works for new movies with no ratings, as long as they have metadata
• Recommendations are explainable: 'because you liked X, which shares tags Y and Z'
• No dependency on other users — privacy preserving
WEAKNESSES:
• Over-specialisation: only recommends things similar to what the user already knows
• Cannot discover cross-genre gems the user would love but wouldn't think to search for
• Quality depends entirely on the richness of the metadata (hence genome tags)
• Cannot capture subjective quality — two films can share all tags but differ vastly in execution

4.2 TF-IDF Vectorization — Deep Dive

TF-IDF (Term Frequency — Inverse Document Frequency) is a classical information retrieval technique that represents each document (here, each movie) as a vector of numerical scores reflecting how characteristic each term is of that document within a collection. It is the workhorse of text-based content similarity.

Before TF-IDF, we need to understand the core problem it solves: if we simply count word occurrences, the word 'the' would dominate every document's representation because it appears everywhere. We need a method that up-weights words that are frequent in a specific document but rare across all documents — those are the words that truly distinguish that document.

4.2.1 Term Frequency (TF)

Term Frequency measures how often a specific term t appears in a specific document d, normalised by the total number of terms in that document. Without normalisation, longer documents would unfairly dominate:

```
TF(t, d) = count(t in d) / total_terms(d)
```

Example: The word 'psychological' appears 4 times in a 50-word soup for 'Black Swan'. TF = 4/50 = 0.08. This tells us 'psychological' is meaningful for this film. But we need to check if 'psychological' is equally common in all other films — if so, it still doesn't distinguish 'Black Swan' from the rest.

CineMatch uses sublinear TF scaling (log normalisation) which further dampens the effect of very high-frequency terms:

```
TF_sublinear(t, d) = 1 + log( count(t in d) )   if count > 0,   else 0
```

This means the difference between a term appearing 1 time versus 10 times counts far less than the difference between 0 times and 1 time. The first occurrence of a term is the most informative; repetitions have diminishing returns.

4.2.2 Inverse Document Frequency (IDF)

IDF measures how rare a term is across the entire collection of documents. A term that appears in almost every document (like 'film' or 'story') conveys very little discriminating information. A term that appears in only a handful of documents (like 'claymation' or 'existentialist') is highly distinctive.

```
IDF(t, D) = log( |D| / |{d ∈ D : t ∈ d}| ) + 1
```

Where |D| is the total number of documents (movies) and the denominator is the number of documents containing term t. The +1 prevents zero-division and ensures terms in every document still get a small positive weight.

Interpretation: If there are 27,000 movies and 'drama' appears in 10,000 of them, IDF('drama') = log(27,000/10,000) + 1 = log(2.7) + 1 ≈ 1.99. If 'claymation' appears in only 50 movies, IDF('claymation') = log(27,000/50) + 1 = log(540) + 1 ≈ 7.29. The rarer term gets a much higher weight.

4.2.3 The TF-IDF Score

The final TF-IDF score for a term t in document d is the product of TF and IDF:

```
TF-IDF(t, d, D) = TF(t, d) × IDF(t, D)
```

A high TF-IDF score means: this term appears frequently in this specific document (high TF) AND it is rare across the collection (high IDF). These are the terms that truly characterise this document versus

all others. A low score means either the term is uncommon in this document, or it is common everywhere, or both.

In practice, each movie is represented as a sparse vector of TF-IDF scores — one dimension per vocabulary term. Most entries are zero (the term does not appear in this movie's soup). The vocabulary is capped at 20,000 terms (max_features), selected by the highest IDF values, effectively removing the most common uninformative terms.

4.2.4 N-Grams: Capturing Phrases

CineMatch uses both unigrams (single words) and bigrams (adjacent word pairs) in its TF-IDF vocabulary. This is controlled by the parameter ngram_range=(1,2).

Without bigrams, the tags 'plot' and 'twist' would be treated as independent terms. A film tagged 'plot twist' would have the same representation as one where these words appear independently in different contexts. With bigrams, 'plot_twist' becomes a single vocabulary term that specifically identifies films with surprising narrative reversals.

Other examples where bigrams are crucial: 'black comedy', 'slow burn', 'feel good', 'martial arts', 'true story', 'based on book'. These compound concepts are fundamentally different from their constituent words.

4.2.5 L2 Normalisation

After computing TF-IDF scores, each movie's feature vector is L2-normalised — divided by its Euclidean norm so that the resulting vector has unit length:

$$v_normalised = v \,/\, ||v||_2 \quad \text{where} \quad ||v||_2 = \text{sqrt}(\, \Sigma_i\, v_i^2 \,)$$

This step is critical for making cosine similarity computable as a simple dot product. After L2 normalisation, all vectors lie on a unit hypersphere, and the dot product between any two vectors equals their cosine similarity directly. This makes the similarity computation at query time extremely fast.

4.3 The Metadata Soup Strategy

Rather than treating each metadata field (genres, tags, etc.) as a separate feature space and combining them with complex feature engineering, CineMatch concatenates all fields into a single text string — the 'metadata soup' — and runs TF-IDF on the whole thing at once.

The key insight is that field importance can be controlled by repetition. If genres should be three times more influential than plot descriptions in determining similarity, simply include the genre text three times in the soup and the text once. TF-IDF naturally amplifies terms that appear more frequently within a document, so repetition directly boosts the weight of those field values.

Field	Weight & Justification
Genome Tags (top 20)	×2 — The most semantically rich signal. Tags like 'psychological thriller', 'nonlinear timeline', and 'bleak ending' capture tone and theme far more precisely than genre labels.

Genres	×3 — The strongest coarse signal. Genre is the primary axis on which user preferences are organised. Someone who dislikes horror almost never wants a horror recommendation regardless of other tags.
User Tags (community)	×1 — Valuable but noisier. Community tags include misspellings, opinions ('masterpiece'), and irrelevancies ('watched with friends'). Lower weight reduces noise impact.

The resulting soup for a film like 'Blade Runner 2049' might look like: 'Sci-Fi Sci-Fi Sci-Fi Mystery Mystery Mystery Drama Drama Drama visually stunning visually stunning dystopia dystopia artificial intelligence artificial intelligence philosophical thought provoking sequel neo-noir ...' — a rich, weighted textual fingerprint.

4.4 Cosine Similarity

After TF-IDF vectorisation and L2 normalisation, every movie is a point on a high-dimensional unit sphere. To measure how similar two movies A and B are, CineMatch uses cosine similarity — the cosine of the angle between their vectors:

```
cosine_similarity(A, B) = A · B / (||A|| × ||B||) = A · B   (after L2 normalisation)
```

The result is a value between -1 and 1. For TF-IDF vectors (which are always non-negative), the result is between 0 and 1. A score of 1.0 means the movies have identical TF-IDF profiles. A score of 0.0 means they share no vocabulary terms at all.

Cosine similarity is preferred over Euclidean distance for text because it is length-invariant. A short genome tag soup (maybe 50 words) and a long one (200 words) can be about the same topic, but their Euclidean distance would be large simply because one vector has larger magnitude. After L2 normalisation, both vectors have magnitude 1, and the angle between them purely reflects directional similarity — i.e., topical similarity.

Geometric Intuition of Cosine Similarity
Imagine all movies as arrows pointing outward from the origin of a 20,000-dimensional space.
Two films about 'psychological crime thrillers' point in roughly the same direction.
A romantic comedy and a war documentary point in nearly opposite directions.
Cosine similarity measures the angle between arrows — not their length.
Small angle (cos ≈ 1.0) = very similar topics.
Right angle (cos ≈ 0.0) = completely unrelated topics.
After L2 normalisation, dot product = cosine — so similarity is O(vocabulary) per query.

4.5 Sparse Nearest-Neighbour at Scale

A naive implementation would precompute and store the full pairwise cosine similarity matrix — a 27,278 × 27,278 grid of float32 values. This requires $27{,}278^2 \times 4$ bytes ≈ 2.97 GB of RAM. On a laptop with 8GB RAM shared with the OS and other processes, this is infeasible.

CineMatch avoids this by computing similarity on demand at query time rather than precomputing. For each recommendation request, it computes the similarity of one query movie against all other movies using a sparse dot product:

```
scores = feature_matrix × query_vector.T   →  shape: (n_movies,)
```

Because the feature matrix is stored as a sparse matrix (CSR format — Compressed Sparse Row), and each row is very sparse (most movies contain only a small fraction of the 20,000 vocabulary terms), this multiplication is extremely fast — comparable to dense matrix multiplication at a fraction of the memory cost.

After computing scores for all movies, the top-N are extracted using numpy.argpartition — an O(n) partial sort that finds the top-N without fully sorting all n elements. This is significantly faster than a full argsort (O(n log n)) for large n.

4.6 Profile-Based Recommendations

When a user specifies multiple liked films (as in the Watchlist tab), CineMatch builds a user taste profile by computing a weighted average of the L2-normalised feature vectors of all liked films:

$$\text{profile_vector} = \Sigma_i \; (w_i \times \text{feature_vector}(\text{movie_}i)) \; / \; \Sigma_i \, w_i$$

Where w_i is the weight for film i — if the user provides explicit ratings, the weights are the normalised rating values (so a 5-star film contributes more than a 3-star film). The profile vector is then re-normalised to unit length and used exactly like a query vector in the standard similarity computation.

This aggregation means the profile represents the user's collective taste — a point in feature space that is a centroid of everything they have liked. Films nearest to this centroid in cosine distance are the recommendations. This is more robust than taking the maximum similarity to any single liked film, which would lock in on the most recent or extreme preference.

5. Collaborative Filtering

5.1 Core Concept

Collaborative filtering recommends items based on the collective behaviour of all users. It operates on a single profound insight: users who agreed strongly in the past are likely to agree in the future. It does not need to know anything about the content of the films — only who rated what and how highly.

The name 'collaborative' reflects that every user implicitly collaborates with every other user by contributing their ratings. A user who rates 30 films provides information not just about those 30 films, but — through the latent structure of the rating matrix — about all 27,000 films they haven't seen yet.

Strengths and Weaknesses of Collaborative Filtering

STRENGTHS:
• Discovers latent preferences the user couldn't articulate — 'I didn't know I liked neo-noir'
• Enables serendipitous cross-genre recommendations based on taste patterns
• Does not require any content metadata — pure behaviour-driven
• Captures quality differences between films with identical content profiles
WEAKNESSES:
• Cold-start: cannot recommend to new users with no rating history
• Cold-item: cannot recommend new movies with no ratings (no matter how good they are)
• Popularity bias: well-rated popular films are easier to model than obscure ones
• Sparsity: the rating matrix is ~98.9% empty — most (user, movie) pairs are unobserved
• Scalability: 138K × 27K is a large matrix to factorise

5.2 The User-Item Rating Matrix

The foundation of collaborative filtering is the user-item matrix R, where each row represents a user, each column represents a movie, and each cell R[u][m] contains user u's rating of movie m (or is empty/zero if they haven't rated it).

Property	Value in CineMatch (full MovieLens 20M)
Dimensions	138,493 users × 27,278 movies = 3.78 billion possible cells
Observed ratings	20,000,263 (after quality filtering: ~14–16 million)
Sparsity	~98.9% of cells are empty — only 1.1% are observed
Rating scale	0.5 to 5.0 in 0.5 increments (9 possible values)
Storage format	scipy CSR (Compressed Sparse Row) — stores only non-zero entries
Memory (full)	~480MB for CSR with float32 values and int32 indices
Memory (dense)	~15GB if stored as a standard float32 2D array — infeasible

The sparse CSR format is essential. Instead of storing all 3.78 billion cells, it stores only the 20 million non-zero values along with their row and column indices — a factor of ~190 reduction in storage. All linear algebra operations on CSR matrices (matrix-vector multiplication, factorisation) are optimised to skip zero entries entirely.

5.3 Matrix Factorization via Truncated SVD

The goal of collaborative filtering is to fill in the missing entries of R — to predict how user u would rate movie m that they haven't seen. This is the matrix completion problem. CineMatch solves it using

Singular Value Decomposition (SVD), the most mathematically principled approach to low-rank matrix approximation.

5.3.1 The Mathematical Foundation

Any matrix R of shape (n_users × n_movies) can be exactly decomposed as:

$$R = U \cdot \Sigma \cdot V^T$$

Where U is an (n_users × n_users) orthogonal matrix, Σ is an (n_users × n_movies) diagonal matrix of singular values in decreasing order, and V^T is an (n_movies × n_movies) orthogonal matrix. The singular values $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma$□ measure the 'importance' of each component.

5.3.2 Truncation — The Low-Rank Approximation

The full SVD uses all singular values and is computationally equivalent to the original matrix. The key insight is that most of the meaningful signal is captured by the largest singular values. The small ones correspond to noise. By keeping only the top k singular values, we get the best rank-k approximation of R:

$$R \approx \hat{R} = U_k \cdot \Sigma_k \cdot V_k^T$$

Where U_k is (n_users × k), Σ_k is (k × k), and V_k is (n_movies × k). CineMatch uses k=100. This means we are approximating a 138K × 27K matrix using only 100 latent dimensions — a dramatic compression that forces the model to learn the most important patterns rather than memorising noise.

5.3.3 What Are Latent Factors?

The k=100 dimensions of the latent space are not labelled or interpretable in advance. They are automatically discovered patterns in the rating data. Conceptually, they might capture things like:

- Dimension 1: preference for cerebral, slow-paced films vs. action-oriented blockbusters
- Dimension 2: preference for classic pre-1980 cinema vs. modern franchises
- Dimension 3: preference for emotionally heavy drama vs. light entertainment
- ... and 97 more subtle, unnamed taste dimensions

In practice, each user is represented as a 100-dimensional vector capturing their position on each of these 100 axes. Each movie is similarly represented. Users who are close together in this latent space have similar tastes. Movies that are close together are liked by similar users — even if they share no genre tags.

This is the power of collaborative filtering: it can discover that users who loved 'Moonlight' also loved 'Call Me By Your Name' — not because these films share obvious metadata, but because a latent taste dimension (perhaps: appreciation of quiet, intimate character studies) connects them in the embedding space.

5.3.4 User and Item Factor Matrices

After factorisation, CineMatch extracts:

- User factors: U_k · Σ_k, shape (n_users, k). Each row is a user's embedding in latent space. Users with similar embeddings have similar tastes.
- Item factors: V_k normalised to unit length, shape (n_movies, k). Each row is a movie's embedding. Movies with similar embeddings are liked by similar users.

Both matrices are stored in memory as dense float32 arrays. At k=100 with 138K users and 27K movies, this requires approximately 100MB of RAM — modest and practical.

5.4 User Mean-Centering

Before passing the rating matrix to SVD, CineMatch applies user mean-centering: subtracting each user's personal mean rating from all their ratings.

```
r_centered(u, m) = r(u, m) - mean_rating(u)
```

This is a critical preprocessing step for two reasons:

- Removes rater bias: Some users give generous ratings (average 4.5/5) and some are harsh critics (average 2.5/5). Without centering, the SVD would spend many of its k latent dimensions capturing this scale difference rather than actual taste differences. After centering, both users' effective neutral point is zero, and positive/negative deviations from zero reflect genuine enthusiasm or dislike.
- Improves factorisation quality: Mean-centred matrices tend to have lower effective rank — the structural signal is cleaner, and SVD can capture more of it in the top k components, leading to higher explained variance.

After centering, the rating matrix is no longer purely non-negative, which is why CineMatch stores it as a signed float32 sparse matrix and uses the sklearn TruncatedSVD (which handles signed values) rather than an NMF (Non-negative Matrix Factorisation) approach.

5.5 Item-Item Latent Similarity

To recommend movies similar to a given movie m, CineMatch computes cosine similarity between m's item factor vector and all other item factor vectors in the latent space:

```
cf_similarity(m, m') = V_k[m] · V_k[m']   (after L2 normalisation of V_k
                                  rows)
```

This is a dense (n_movies, k) × (k,) dot product, producing a score for every movie in the catalog. Because k=100 and n_movies ≈ 27K, this is a very fast operation (2.7 million multiplications, done in a single numpy call).

The scores reflect collaborative similarity: films that are close in latent space are liked by similar users, even if they share no genre, director, or any other explicit attribute. This is what allows collaborative filtering to surface unexpected but appropriate recommendations.

5.6 Cold-Start Handling

When a user has no rating history in the dataset (pure cold start), the collaborative model has no user-specific embedding to work with. CineMatch handles this in two ways:

- Pseudo-user vector from liked films: Given a list of films the user explicitly marks as liked (e.g. in the Watchlist tab), CineMatch constructs a pseudo-user embedding as a weighted average of the item factor vectors of those films. This pseudo-embedding is then used exactly like a real

user embedding to score all other movies. It is not as accurate as a learned embedding from actual ratings, but it is far better than a cold-start fallback to popularity.

- Content-model fallback: If no liked films are provided, the hybrid system falls back entirely to content-based recommendations (alpha=1.0), which requires no user history at all.

6. Hybrid Recommendation Engine

6.1 Why Neither Approach Alone Is Sufficient

Content-based and collaborative filtering have complementary failure modes. A system that uses only one will systematically fail in ways that the other would not. The hybrid approach is not merely a performance optimisation — it is a fundamental robustness requirement.

Scenario	Content-Based	Collaborative
New user, no history	Works (uses liked films)	Fails (no embedding)
New movie, no ratings	Works (uses metadata)	Fails (no item embedding)
Niche film, few ratings	Works (genome tags exist)	Weak (sparse signal)
Popular film, many ratings	Works but may over-fit genre	Strong (dense signal)
Cross-genre serendipity	Fails (stays within genre)	Works (latent taste dims)
Quality distinction within genre	Fails (same tags = same sim)	Works (ratings reflect quality)

The table shows that content-based and collaborative filtering fail in precisely opposite situations. By blending them, CineMatch is robust to all six scenarios simultaneously.

6.2 Score Normalisation

Content scores and CF scores are on completely different numerical scales. Content scores are cosine similarities bounded to [0, 1]. CF scores are dot products of L2-normalised item embeddings, also in [0, 1] but with a different distribution of values. Simply adding them would give disproportionate weight to whichever happens to have larger raw values.

CineMatch normalises both score sets to [0, 1] using min-max normalisation independently before blending:

```
score_normalised = (score - score_min) / (score_max - score_min)
```

This maps the lowest-scoring candidate in each model's output to 0 and the highest to 1, making the scales fully comparable before the hybrid blend is applied. The normalisation is applied separately to the content and CF candidate pools.

6.3 Weighted Hybrid Blend Formula

The central formula of the hybrid engine combines three normalised signals:

```
hybrid_score(m) = α × content_score_n(m) + (1−α) × cf_score_n(m) + β × pop_score_n(m)
```

Where:

- α ∈ [0, 1] is the content-collaborative balance weight, configurable in real time via the sidebar slider.
- (1−α) is the complementary collaborative weight. When α=0, the system is pure collaborative. When α=1, it is pure content-based.
- β = 0.1 is a fixed small weight for the popularity signal, providing a gentle bias toward well-regarded films.
- pop_score_n is the normalised Bayesian popularity score (combining Bayesian rating and log vote count).

The outer merge strategy ensures that candidates from either model — even those appearing in only one of the two candidate pools — are included in the final ranking. A film that scores very high on content similarity but has no CF signal gets cf_score_n = 0. A film with strong CF signal but no content match gets content_score_n = 0. The hybrid blend then naturally weights each film according to how much evidence each model provides.

Alpha Setting	System Behaviour
α = 1.0 (pure content)	Recommendations driven entirely by genome tag and genre similarity. Best for new users or when collaborative signal is absent. Stays safely within familiar territory.
α = 0.7 (content-lean)	Mostly driven by content, with a moderate CF influence. Good for users with a clear genre preference who still want some collaborative serendipity.
α = 0.5 (balanced default)	Equal blend of both signals. The recommended starting point for most users. Balances familiar territory with discovery.
α = 0.3 (CF-lean)	Mostly driven by collaborative patterns. Good for active users with rich rating history. More surprising recommendations.
α = 0.0 (pure CF)	Recommendations driven entirely by latent taste patterns. Can surface unexpected gems but may miss obvious content matches. Best for highly active users.

6.4 Popularity Signal & Bayesian Boost

The popularity component of the hybrid score is itself a blend of two signals:

```
pop_score(m) = 0.6 × normalise(bayesian_rating(m)) + 0.4 × normalise(log(1 + rating_count(m)))
```

The 0.6/0.4 split gives slightly more weight to quality (Bayesian rating) than raw popularity (vote count). The log transformation of vote count is essential: without it, a blockbuster with 100,000 ratings would score 1000× higher on popularity than a critically acclaimed film with 100 ratings. The log reduces this to log(100,001)/log(101) ≈ 11.5/4.6 ≈ 2.5× — a much more proportional representation of genuine popularity.

The β=0.1 overall weight for the popularity signal is deliberately small. Its purpose is not to override relevance but to break ties between equally similar candidates in favour of films the user is more likely to have heard of, and to guard against completely obscure recommendations that happen to score well on sparse data.

7. MMR Diversity Re-Ranking

7.1 The Filter Bubble Problem

Consider a user who has just watched 'The Godfather' and asks for similar movies. A pure relevance-maximising system might return: Goodfellas, The Departed, Casino, Heat, Scarface, Donnie Brasco, Carlito's Way, American Gangster. All are mafia/crime dramas — an obvious and accurate list, but also entirely predictable. The user could have found this list themselves. There is no discovery.

Worse, after consuming this list, the user's profile is now saturated with crime drama signals. Future recommendations will be even more narrowly crime-focused. This positive feedback loop is the filter bubble — the system converges on an ever-narrowing slice of the user's taste, ignoring the full breadth of what they might enjoy.

A better list for 'The Godfather' might be: Goodfellas (highest relevance crime drama), Schindler's List (historical epic drama, different setting), The Shawshank Redemption (character study, prison), Apocalypse Now (morally complex, dark tone), The Dark Knight (organised crime themes, modern), Parasite (class and power dynamics), No Country for Old Men (violent, moral ambiguity), There Will Be Blood (power and corruption). Each film shares something with The Godfather, but together they cover a wide range of genres and eras.

7.2 Maximal Marginal Relevance — Theory

Maximal Marginal Relevance (MMR), introduced by Carbonell and Goldstein in their 1998 paper 'The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries', is a greedy algorithm that balances relevance with novelty relative to already-selected items.

At each step, for each remaining candidate film d, MMR computes a score that rewards high relevance to the query but penalises similarity to films already selected:

```
MMR(d) = (1 − λ) × relevance(d)  −  λ × max_{s ∈ Selected} sim(d, s)
```

Where:

- relevance(d) is the hybrid_score — how good a recommendation film d is on pure relevance grounds.
- sim(d, s) is the similarity between candidate d and an already-selected film s — measured using Jaccard genre similarity.
- max_{s ∈ Selected} sim(d, s) is the maximum similarity to any already-selected film — the most relevant measure of redundancy.
- λ ∈ [0, 1] controls the diversity-relevance trade-off. CineMatch uses λ = 0.3.

7.3 Jaccard Genre Similarity

For the similarity measure within MMR, CineMatch uses Jaccard similarity between the genre sets of two films. The Jaccard index measures the overlap between two sets as a fraction of their union:

```
Jaccard(A, B) = |genres(A) ∩ genres(B)| / |genres(A) ∪ genres(B)|
```

Example: Film A has genres {Crime, Drama, Thriller}. Film B has genres {Crime, Drama}. Intersection = {Crime, Drama}, size 2. Union = {Crime, Drama, Thriller}, size 3. Jaccard similarity = 2/3 = 0.667.

If Film C has genres {Animation, Children, Comedy}, then Jaccard(A, C) = |{}|/|{Crime, Drama, Thriller, Animation, Children, Comedy}| = 0/6 = 0.0. Films A and C are completely dissimilar by genre, so selecting A does not penalise selecting C — it is a genuinely novel addition to the list.

Jaccard is used rather than cosine similarity over genre vectors because: (1) genre sets are binary (a film either has a genre or not, no continuous values), (2) Jaccard naturally handles sets of different sizes without normalisation issues, (3) it is extremely fast to compute.

7.4 MMR Algorithm Step-by-Step

The MMR algorithm in CineMatch runs as follows on a pool of candidate films sorted by hybrid_score:

1. Initialise: Selected = [] (empty list). Remaining = all candidate films sorted by hybrid_score.
2. First selection: Add the highest-scoring candidate to Selected (no redundancy penalty yet, since Selected is empty). Remove from Remaining.
3. Subsequent selections: For each film d still in Remaining, compute MMR(d) = (1 − 0.3) × hybrid_score(d) − 0.3 × max(Jaccard(d, s) for s in Selected).
4. Select the film d* with the highest MMR score. Add to Selected, remove from Remaining.
5. Repeat steps 3–4 until |Selected| = top_n.

The time complexity is O(top_n × |candidates|) — linear in both the number of results requested and the candidate pool size. For typical values (top_n=12, candidates=80), this is 960 MMR computations, negligible in practice.

MMR In Practice: 'The Godfather' Example
Step 1: Goodfellas selected (hybrid_score=0.95, highest relevance, Jaccard to self=N/A)
Step 2: The Departed would score high relevance but Jaccard(The Departed, Goodfellas)=0.75
MMR(The Departed) = 0.7×0.88 − 0.3×0.75 = 0.616 − 0.225 = 0.391
Schindler's List: MMR = 0.7×0.76 − 0.3×0.33 = 0.532 − 0.099 = 0.433 ← selected
Step 3: Now Selected={Goodfellas, Schindler's List}. The Departed now penalised by both.
Each step naturally diversifies: after several crime films are selected,
further crime films incur high penalties and non-crime films are favoured.
Result: A list that is both individually relevant AND collectively diverse.

8. System Architecture & File Structure

8.1 Module Overview

CineMatch is organised as a clean layered architecture. Each module has a single responsibility and communicates through well-defined pandas DataFrame interfaces. No module depends on the UI layer; all models can be used independently from Python scripts.

Module / File	Responsibility & Key Functions
data/loader.py	All data I/O and preprocessing. find_data_dir() auto-detects CSV location. load_movies() parses genres and extracts year. load_ratings() chunks 20M rows with quality filters and Parquet caching. load_genome() selects top-1000 tags. build_movie_tag_text() assembles the weighted metadata soup. compute_movie_stats() adds Bayesian ratings.
models/content_based.py	ContentBasedModel class. fit() builds TF-IDF and L2-normalises the sparse feature matrix. get_similar_movies() runs a sparse dot product for one query film. get_profile_recommendations() averages feature vectors of liked films. get_top_terms() returns the highest TF-IDF vocabulary terms for a film (for explainability).
models/collaborative.py	CollaborativeModel class. fit() builds the CSR user-item matrix with mean-centering and runs TruncatedSVD. get_similar_movies() computes item-item latent similarity. get_user_recommendations() predicts ratings for all unseen films for a known user. get_profile_recommendations() builds a pseudo-user embedding from liked films.
models/hybrid.py	HybridRecommender class. _merge_scores() outer-joins content and CF candidates and applies min-max normalisation. _mmr_rerank() runs the greedy Jaccard-penalised re-ranking. recommend_for_movie() and recommend_for_profile() are the primary public API endpoints. get_trending() and search_movies() support browse and search.
utils/evaluation.py	Offline evaluation framework. Implements all 8 metrics as pure functions. evaluate() orchestrates the 80/20 hold-out split across n_users sampled users and returns mean ± std for each metric.
pipeline.py	End-to-end build function. build_pipeline() chains all loading, enrichment, and fitting steps in the correct order. Accepts a status_callback for Streamlit progress messages. Returns (recommender, movies_df, ratings_df) ready for use.
app.py	Streamlit UI. Loads the pipeline via @st.cache_resource (fits once per session). Provides 5 tabs: Similar Movies, My Watchlist, Trending & Browse, Analytics (Plotly charts), and Evaluation (radar chart + metrics table). All recommendation controls are live-updating sidebar widgets.

8.2 Data Flow Diagram

The end-to-end data flow through the system proceeds in the following order:

6. movie.csv → load_movies() → movies_df [27K rows, genres parsed, year extracted]
7. rating.csv → load_ratings() → ratings_df [~16M rows after filtering, Parquet cached]
8. genome-scores.csv + genome-tags.csv → load_genome() → genome_scores, genome_tags
9. tag.csv → load_user_tags() → user_tags_df [community tags]
10. movies_df + ratings_df → compute_movie_stats() → movies_df enriched with mean_rating, rating_count, bayesian_rating
11. movies_df + genome_scores + genome_tags + user_tags_df → build_movie_tag_text() → movies_df.tag_soup [weighted text soup per movie]
12. movies_df.tag_soup → ContentBasedModel.fit() → sparse TF-IDF feature matrix (27K × 20K)
13. ratings_df + movies_df → CollaborativeModel.fit() → user_factors (138K × 100) + item_factors (27K × 100)
14. ContentBasedModel + CollaborativeModel → HybridRecommender [ready to serve queries]
15. Query → HybridRecommender → content candidates + CF candidates → merge → normalise → blend → MMR → top-N results

8.3 Memory Budget Estimate

Component	Approximate RAM (full MovieLens 20M, k=100)
ratings_df (filtered, float32)	~640 MB (16M rows × 4 columns × 4 bytes)
Sparse CSR user-item matrix	~480 MB (16M non-zeros × 12 bytes each)
TF-IDF feature matrix (sparse, 27K×20K)	~200 MB (sparse, ~1% non-zero density)
User latent factors (138K × 100)	~55 MB (float32)
Item latent factors (27K × 100)	~11 MB (float32)
movies_df with all enrichment columns	~200 MB (genome text, tags, stats)
Streamlit overhead + Python runtime	~500 MB
TOTAL (approximate)	~2.1 GB — fits in 8GB RAM with headroom

9. Streamlit User Interface

9.1 Design Philosophy

The UI follows a 'cinematic dark' design language: deep navy backgrounds, warm gold accent colours, crimson highlights, and serif typography (Playfair Display for titles, DM Sans for body text). This aesthetic matches the domain — a premium movie discovery experience — while being functional and information-dense.

All recommendation controls are placed in the sidebar and take effect immediately without page reloads. The alpha slider, top-N control, diversity toggle, and minimum-votes filter all feed directly into the recommendation engine on every query. This allows users to explore how changing the content-collaborative balance affects their results in real time — a genuinely interactive ML experience.

9.2 The Five Tabs

Tab	Features & Implementation
Similar Movies	Text search box calls recommender.search_movies() for autocomplete-style filtering of the 27K catalog. Selected film is displayed with its genome tags, Bayesian rating, and vote count. Recommendations show match score bar, top genome tags, genres, and metadata. The top-N TF-IDF terms for the selected film are shown for explainability.
My Watchlist	Multi-select of up to 15 films from the catalog. Auto-infers a taste profile (genre frequency across all selections). Calls recommend_for_profile() with uniform 5.0 weights. Results are shown in a two-column card grid for efficient browsing. Taste profile genre badges update immediately as films are added or removed.
Trending & Browse	Three modes: All-Time Best (Bayesian rating with min-votes filter), By Genre (multi-select genre filter), and By Decade (slider from 1920s to 2020s). All backed by get_trending() with appropriate filters. Two-column card layout. The min-votes filter from the sidebar propagates here, letting users explore both mainstream and niche films.
Analytics	Five KPI tiles (movie count, users, ratings, avg rating, sparsity). Genre distribution horizontal bar chart with colour gradient. Rating distribution histogram (sampled to 200K for speed). Ratings-per-user histogram showing the long-tail distribution. Films-by-decade combo chart (bar for count, line for avg rating). All charts are Plotly with a matching dark theme.
Evaluation	Architecture summary cards for all three model components with live parameter values. Configurable evaluation (50–500 users). After running, displays a Plotly radar chart of all 8 metrics plus a detailed table with mean ± std for each metric. Explains the hold-out protocol and what each metric means for the non-expert user.

9.3 Performance Optimisations in the UI

- @st.cache_resource: The pipeline (data loading + model fitting) runs once per session and is cached. Subsequent tab switches and control changes do not refit the models.
- @st.cache_data: The evaluation results are cached separately so re-running evaluation on the same dataset does not repeat the computation.
- Status callback: The pipeline.py build_pipeline() function accepts a status_callback parameter. The UI passes a Streamlit placeholder that updates with progress messages during the 1–15 minute build process, so the user sees live status rather than a frozen spinner.
- Sampled charts: The rating histogram samples 200,000 ratings rather than plotting all 20M, giving identical visual information in milliseconds.

10. Evaluation Framework

10.1 Offline Hold-Out Protocol

CineMatch uses a hold-out evaluation protocol — the standard methodology for offline recommender system evaluation. For each sampled user, their complete rating history is split into a training set (80%) and a test set (20%). The test set acts as ground truth: films the user has actually rated highly represent what they 'truly' enjoy. The system then generates recommendations using only the training information and measures how well those recommendations align with the test set.

Hold-Out Protocol Details
Users sampled: n_users (configurable, default 200) selected at random
Minimum history: Users with fewer than 10 ratings are skipped
Split ratio: 80% train / 20% test (random, stratified by user)
Relevance threshold: test items rated ≥ 4.0 out of 5.0 are considered 'relevant'
Minimum relevance: If a user has fewer than 2 relevant test items, they are skipped
Profile input: Training items rated ≥ 4.0 are used as the 'liked films' profile
Evaluation mode: diversity=False (MMR off) — we measure relevance, not diversity
Reported values: Mean and standard deviation across all evaluated users

The hold-out protocol simulates real-world use: we pretend we don't know about the test ratings, generate recommendations, then check whether those recommendations include films the user actually liked. This is an imperfect proxy for true recommendation quality (users may enjoy recommended films they haven't yet seen) but is the best offline evaluation possible without a live A/B test.

10.2 All Eight Metrics Explained

Metric 1: Precision@K

Precision@K measures the fraction of the top-K recommended films that are actually relevant to the user:

```
Precision@K = |Relevant ∩ Top-K Recommendations| / K
```

If K=10 and 2 of the 10 recommendations are films the user rated ≥ 4.0 in the test set, Precision@10 = 2/10 = 20%. Precision directly answers: 'Of every 10 recommendations, how many are good ones?' It does not care about where in the list the relevant items appear.

Metric 2: Recall@K

Recall@K measures the fraction of all relevant items in the test set that appear in the top-K recommendations:

```
Recall@K = |Relevant ∩ Top-K Recommendations| / |Relevant|
```

If a user has 8 films rated ≥ 4.0 in their test set and 3 of them appear in the top-10 recommendations, Recall@10 = 3/8 = 37.5%. Recall answers: 'What fraction of the things the user would love did we successfully find?' Precision and Recall trade off: a larger K improves recall but worsens precision.

Metric 3: F1@K

F1@K is the harmonic mean of Precision@K and Recall@K — a single number that balances both:

```
F1@K = 2 × Precision@K × Recall@K / (Precision@K + Recall@K)
```

The harmonic mean (rather than arithmetic mean) penalises extreme imbalances. A system with Precision=100% and Recall=1% would have arithmetic mean 50.5% but harmonic mean (F1) of only 2% — correctly reflecting that finding almost nothing relevant is a poor system regardless of precision.

Metric 4: NDCG@K (Normalised Discounted Cumulative Gain)

NDCG is the most sophisticated of the ranking metrics. It rewards systems not just for finding relevant items, but for placing them high in the ranked list. A relevant item at position 1 is worth much more than one at position 10.

Discounted Cumulative Gain (DCG) is computed by summing the relevance of each position, discounted logarithmically by its rank:

$$DCG@K = \Sigma_{i=1}^{K} relevance(i) / \log_2(i + 1)$$

A relevant item at rank 1 contributes $1/\log_2(2) = 1.0$. At rank 2: $1/\log_2(3) = 0.63$. At rank 10: $1/\log_2(11) = 0.29$. The logarithmic discount ensures that getting results right matters most near the top of the list, where users actually look.

Normalisation divides DCG by the ideal DCG (all relevant items at the top of the list), giving a score from 0 to 1 regardless of how many relevant items exist:

```
NDCG@K = DCG@K / IDCG@K
```

Metric 5: MRR (Mean Reciprocal Rank)

MRR focuses on the single most important question: where does the first relevant item appear? It computes the reciprocal of the rank of the first hit:

```
RR(u) = 1 / rank_of_first_relevant_item    (0 if none in top-K)

MRR = mean( RR(u) ) across all evaluated users
```

If the first relevant item is at rank 1, RR = 1.0. At rank 5, RR = 0.2. At rank 10, RR = 0.1. Not found: RR = 0. MRR is particularly relevant for use cases where the user only looks at the very top of the list — it measures whether the system gets at least one right answer close to the top.

Metric 6: Hit Rate@K

Hit Rate@K is the simplest binary metric: did the recommendation list contain at least one relevant item?

```
Hit(u) = 1  if any relevant item appears in top-K,  else 0

Hit Rate@K = mean( Hit(u) ) across all users
```

Hit Rate@10 of 45% means that for 45% of users, at least one of their top-10 recommendations was a film they actually rated highly in the test set. This is a user-satisfaction-oriented metric: even a single good recommendation in a list of 10 is a successful user experience.

Metric 7: Intra-List Diversity

Diversity measures how different the recommended films are from each other — the complement of the filter bubble problem. CineMatch computes average pairwise Jaccard genre dissimilarity across all pairs in the recommendation list:

```
Diversity = mean( 1 - Jaccard(A, B) )  for all pairs (A,B) in top-K
```

A diversity score of 1.0 means every pair of recommendations shares no genres at all — maximum variety. A score of 0.0 means all recommendations are in identical genres. In practice, healthy diversity scores are in the range 0.6–0.8, indicating that the list covers several different genre areas while maintaining overall relevance.

Metric 8: Novelty

Novelty measures how non-mainstream the recommendations are — whether the system is recommending films the user could easily have found themselves (popular blockbusters) or genuinely surfacing lesser-known titles:

```
Novelty = mean( -log2( rating_count(m) / max_rating_count ) )  for m in
                                    top-K
```

A film with 1 million ratings scores low novelty (it's everywhere). A film with 1,000 ratings scores high novelty (it's niche). The logarithm gives the measure an information-theoretic interpretation: it represents the average self-information (surprise) of a recommendation. Higher novelty means the system is doing more 'work' — it's not just recommending obvious popular films.

10.3 Expected Results & Benchmarks

Expected metric ranges for CineMatch on the full MovieLens 20M dataset (k=10 recommendations, evaluated on 200 users):

Metric	Expected Range	State-of-Art*	Key Driver
Precision@10	10% – 20%	25% – 40%	Collaborative signal quality, k factors
Recall@10	15% – 35%	20% – 40%	Catalog coverage, user history length
F1@10	12% – 25%	20% – 35%	Balance of both above
NDCG@10	15% – 30%	30% – 50%	Ranking quality, SVD k value
MRR@10	10% – 25%	25% – 45%	First-hit position in ranked list
Hit Rate@10	35% – 60%	55% – 75%	Probability of any hit in top-10
Diversity	60% – 80%	N/A	MMR lambda parameter
Novelty	0.8 – 2.0	N/A	Popularity weight in hybrid score

*State-of-art figures are for deep learning models (Neural CF, Two-Tower) on MovieLens 1M/20M. CineMatch's SVD approach is competitive with classical baselines and significantly faster to train, with no GPU requirement.

11. Performance, Memory & Scalability

11.1 Loading & Training Times

Step	Full 20M	Dev Mode (20K users)
rating.csv chunked load	~45 sec	~8 sec
Quality filtering	~20 sec	~3 sec
Parquet cache write	~10 sec	~2 sec
Parquet cache read (subsequent)	~3 sec	~1 sec
genome-scores.csv load	~30 sec	~30 sec (unchanged)
build_movie_tag_text()	~15 sec	~15 sec (all movies)
TF-IDF fit (27K movies, 20K vocab)	~20 sec	~20 sec (all movies)
TruncatedSVD fit (k=100)	~8–12 min	~3–5 min
Total cold start	~15 min	~5 min
Total with Parquet cache	~13 min	~4 min

11.2 Query Latency

Once the pipeline is loaded, individual recommendation queries are very fast because all expensive operations (matrix factorisation, TF-IDF fitting) are done at load time, not query time:

Query Type	Typical Latency
Content similarity (1 film)	~5–20 ms (sparse dot product, 27K movies, 20K features)
CF similarity (1 film)	~1–3 ms (dense dot product, 27K movies, 100 features)
Outer merge + normalise + blend	~5–10 ms (pandas merge on candidate pool ~80 films)
MMR re-ranking (80 candidates→12)	~2 ms (O(12×80) Jaccard computations)
Total recommendation query	~15–35 ms (well within interactive latency budget)
Profile recommendation (15 films)	~50–100 ms (15× TF-IDF lookups + CF lookups)

Trending query (sorted by bayesian)	~5 ms (single pandas nlargest())
Title search	~3 ms (str.contains on 27K titles)

11.3 Scaling Beyond 20M Ratings

Several architectural choices position CineMatch for scaling beyond MovieLens 20M:

- Chunked loading: The chunk-based rating loader works on any size file. For a hypothetical 200M rating dataset, increase chunk_size and add more RAM.
- Sparse matrices: scipy CSR scales linearly with non-zero entries, not matrix dimensions. Doubling ratings doubles memory but not query time.
- On-demand similarity: Avoiding the precomputed full similarity matrix means memory usage is independent of n_movies^2. Adding 100,000 more movies increases feature matrix memory linearly, not quadratically.
- Approximate nearest neighbours (ANN): For truly large-scale systems (millions of movies), the exact sparse dot product at query time can be replaced with FAISS or ScaNN ANN indices, trading a small accuracy loss for orders-of-magnitude speed improvement.
- Implicit ALS: The current TruncatedSVD can be replaced with the implicit library's ALS (Alternating Least Squares) implementation which is specifically optimised for large implicit feedback datasets and supports GPU acceleration.

12. Limitations & Future Work

12.1 Current Limitations

Limitation	Description & Impact
Static model (no online learning)	The SVD factors are computed once at startup and do not update as new ratings arrive. In production, a model becomes increasingly stale over time. Periodic retraining (nightly batch jobs) or online matrix factorisation would be needed.
No temporal dynamics	All ratings are treated equally regardless of their timestamp. A user's 2024 ratings should carry more weight than their 2005 ratings — tastes evolve. Time-weighted rating matrices or session-based models would improve recency sensitivity.
No explicit user accounts in UI	The Watchlist tab uses in-session state rather than persistent user profiles. In production, users would need accounts so their rating history persists across sessions.
Cold-item problem	New films with no genome scores (the genome was generated at a fixed point in time) cannot benefit from the rich tag-based content model. They would need manual genre tagging only, which is much coarser.
No implicit feedback	The system only uses explicit star ratings. In practice, implicit signals (watches, clicks, completions, rewinds) are far more abundant and often more informative. The system ignores these.

Genome coverage	The genome covers ~10,381 of 27,278 movies. The remaining ~17,000 movies rely only on genre tags for content-based similarity, which is significantly coarser.

12.2 Recommended Future Enhancements

- Neural Collaborative Filtering (NCF): Replace SVD with a two-layer MLP that maps user and item embeddings to predicted ratings. NCF consistently outperforms SVD on standard benchmarks and can be fine-tuned incrementally as new ratings arrive.
- Two-Tower Architecture: Train separate neural encoders for users and items, then score by dot product. This is the architecture used in production at YouTube, Spotify, and Pinterest. The item encoder can incorporate both content features (genome tags) and collaborative signals.
- Session-based recommendations: Add a recency-aware sequence model (GRU4Rec or SASRec) that models the user's within-session behaviour — the last 5 films watched — for real-time contextual recommendations.
- TMDB API integration: Fetch rich metadata (poster images, cast photos, synopsis, budget, box office) from TMDB using the links.csv cross-reference. This would enable image-based UI cards and richer content features.
- Implicit ALS: Replace the explicit rating matrix with an implicit feedback matrix (view counts, completion rates), using the weighted ALS algorithm which is specifically designed for this signal type.
- A/B testing infrastructure: Implement a logging layer that records which recommendations were clicked or watched, enabling online evaluation and continuous improvement of the alpha parameter and MMR lambda based on real user behaviour.
- Temporal splits: Replace random 80/20 splits with temporal splits (train on ratings before date X, test on ratings after date X), which better simulates the real-world deployment scenario and avoids data leakage through timestamp.

13. Conclusion

CineMatch represents a complete, production-conscious hybrid recommendation system built on the MovieLens 20M dataset. It demonstrates that classical machine learning techniques — TF-IDF and Truncated SVD — applied with engineering care and domain understanding, can produce genuinely useful recommendations at scale without requiring deep learning or GPU hardware.

The system's key contributions beyond a baseline implementation are fourfold. First, genome tag enrichment elevates the content model from coarse genre matching to nuanced semantic similarity — the difference between recommending 'another crime film' and recommending 'another psychologically complex crime film with a non-linear narrative.' Second, user mean-centring in the collaborative model removes rater bias and focuses latent factors on taste patterns rather than scale artefacts. Third, Bayesian rating smoothing ensures that popularity rankings reflect genuine quality rather than sample size effects. Fourth, MMR diversity re-ranking breaks the filter bubble without sacrificing overall relevance, producing recommendation lists that are individually relevant and collectively varied.

The system is designed to be extended. The modular pipeline architecture means any component — the content model, the collaborative model, the blending strategy — can be replaced independently

without touching the rest of the codebase. The alpha slider, MMR lambda, and latent factor count k are all exposed as configurable parameters rather than hardcoded constants, enabling systematic hyperparameter optimisation as the system matures.

Final System Summary	
Dataset	: 20,000,263 ratings · 138,493 users · 27,278 movies (MovieLens 20M)
Content Model	: TF-IDF (1–2 grams, 20K vocab, sublinear TF) on genome-enriched soup
Collab Model	: TruncatedSVD (k=100, mean-centered) on 138K × 27K sparse CSR matrix
Hybrid Blend	: α·content + (1−α)·CF + 0.1·Bayesian_popularity (configurable α)
Diversity	: Greedy MMR with Jaccard genre penalty (λ=0.3, pool=80, top-N=12)
Data Engineering	: 2M-row chunks · int32/float32 dtypes · Parquet caching · quality filters
Evaluation	: 8 metrics: P@K, R@K, F1, NDCG, MRR, Hit Rate, Diversity, Novelty
Query Latency	: 15–35ms per recommendation query after pipeline load
RAM Footprint	: ~2.1 GB for full 20M dataset · fits in 8GB laptop
UI	: 5-tab Streamlit app with real-time controls, Plotly analytics, eval radar

To run the system: install requirements.txt, place MovieLens 20M CSV files in a folder, launch 'streamlit run app.py', and set the folder path in the sidebar. The pipeline builds automatically on first launch and caches for fast restarts thereafter.